

MySQL CASE Statements

By mastering CASE statements, you can write more efficient and readable SQL queries, handling complex business logic directly in your database operations.

Example 1: Using CASE in SELECT

The following example categorizes students based on their grades:

```
## Assign grades based on percentages
SELECT name, grade,
CASE
  WHEN grade >= 90 THEN 'A'
  WHEN grade >= 80 THEN 'B'
  WHEN grade >= 70 THEN 'C'
  WHEN grade >= 60 THEN 'D'
  ELSE 'F'
END AS letter_grade
FROM students;
```

Example 2: Using CASE in an UPDATE

Updating salaries based on employee roles:

```
## Assign raise amounts of 10% for Managers, 5% for Developers, and 2% for other roles
UPDATE employees
SET salary = CASE
  WHEN role = 'Manager' THEN salary * 1.10
  WHEN role = 'Developer' THEN salary * 1.05
  ELSE salary * 1.02
END;
```

Example 3: Using CASE with Aggregate Functions

Counting the number of employees in different salary ranges:

```
## Get a count of employees in each salary range
SELECT
  COUNT(CASE WHEN salary > 50000 THEN 1 END) AS high_salary,
  COUNT(CASE WHEN salary BETWEEN 30000 AND 50000 THEN 1 END) AS mid_salary,
  COUNT(CASE WHEN salary < 30000 THEN 1 END) AS low_salary
FROM employees;
```

Example 4: Multiple Conditions

```
## Assign a salary category to each employee based on their salary range
SELECT employee_name,
CASE
  WHEN salary >= 60000 AND years_of_service >= 10 THEN salary * .25 -- Full Bonus
  WHEN salary >= 60000 AND years_of_service < 10 THEN salary * .15 -- Partial Bonus
  WHEN salary < 60000 AND years_of_service >= 10 THEN salary * .10 -- Loyalty Bonus
  ELSE 'No Bonus'
END AS bonus_amount
FROM employees;
```

Example 5: Using CASE in ORDER BY

CASE statements can also be used in ORDER BY clauses to create custom sorting logic:

```
## Categorize the quantities but also sort the results based on these categories
SELECT OrderID, Quantity,
CASE
  WHEN Quantity > 30 THEN 'The quantity is greater than 30'
  WHEN Quantity = 30 THEN 'The quantity is 30'
  ELSE 'The quantity is under 30'
END AS QuantityText
FROM OrderDetails
ORDER BY
CASE
  WHEN Quantity > 30 THEN 1
  WHEN Quantity = 30 THEN 2
  ELSE 3
END;
```

Best Practices

1. Always include an ELSE clause to handle unexpected cases.
2. Use CASE statements to simplify complex conditional logic in your queries.
3. Remember that CASE statements evaluate conditions in order and stop at the first true condition.
4. CASE statements can be used in SELECT, WHERE, ORDER BY, and UPDATE clauses, offering great flexibility in query design.